POSTGRESQL DDL

ROcodeFest schema

Operational (3NF) schema plus the analytics star schema powering the partner dashboard and leaderboard. Each domain opens with an ER diagram of how its tables connect, then the DDL. Full design rationale lives as comments in the source .sql files.

OLTP · 3NF Analytics · star schema

`db/01_oltp_schema.sql · db/02_analytics_schema.sql`

OLTP SCHEMA 10 domains

Whole schema, as a tree

Every table in 01_oltp_schema.sql, grouped under its domain — pure containment, no FK arrows, so nothing can cross or overlap. The FK relationships within and across these domains are drawn in the per-section diagrams below.

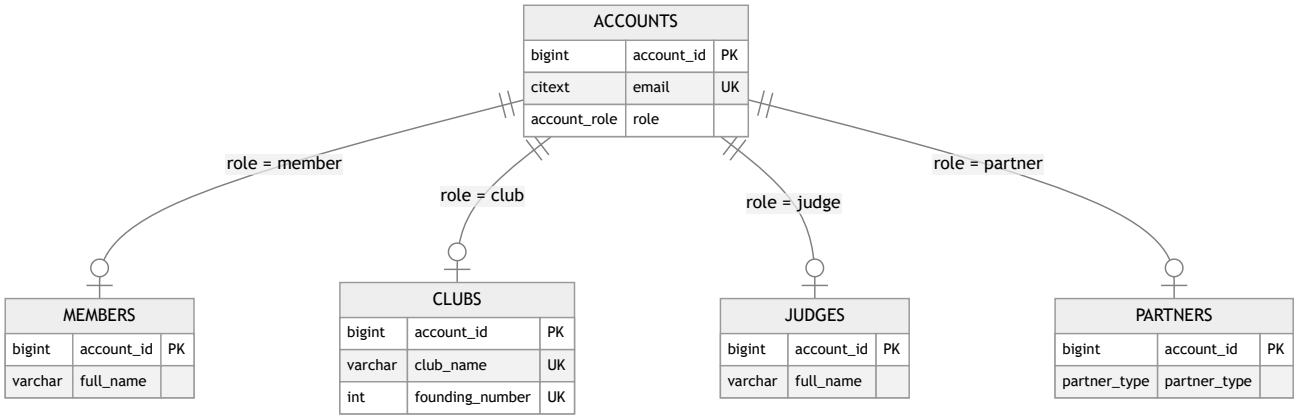


1 Extensions & Types — citext for case-insensitive email; closed vocabularies as ENUMs.

```
CREATE EXTENSION IF NOT EXISTS citext; -- case-insensitive email uniqueness

CREATE TYPE account_role AS ENUM ('member', 'club', 'judge', 'partner');
CREATE TYPE partner_type AS ENUM ('sponsor', 'speaker', 'mentor', 'vendor', 'other');
CREATE TYPE request_status AS ENUM ('pending', 'accepted', 'rejected'); -- shared: join_requests + appeals
CREATE TYPE challenge_stage AS ENUM ('city_challenge', 'final');
CREATE TYPE match_status AS ENUM ('scheduled', 'in_progress', 'completed', 'cancelled');
CREATE TYPE certificate_type AS ENUM ('participation', 'winner', 'runner_up', 'special_award');
CREATE TYPE notification_type AS ENUM (
  'join_request_received', 'join_request_accepted', 'join_request_rejected',
  'appeal_accepted', 'appeal_rejected',
  'announcement_posted',
  'comment_posted', 'reply_posted',
  'submission_assigned', 'judge_feedback_given'
);
```

2 Accounts & Roles — One login table; a subtype table per role (table-per-subtype).



One account, one of four optional role extensions — whichever matches accounts.role.

```sql
CREATE TABLE accounts (
  account_id     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email          CITEXT NOT NULL,
  password_hash  TEXT NOT NULL,
  role           account_role NOT NULL,
  is_active      BOOLEAN NOT NULL DEFAULT TRUE,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_accounts_email UNIQUE (email)
);

CREATE TABLE members (
  account_id  BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  full_name   VARCHAR(150) NOT NULL,
  city        VARCHAR(120),
  bio         TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

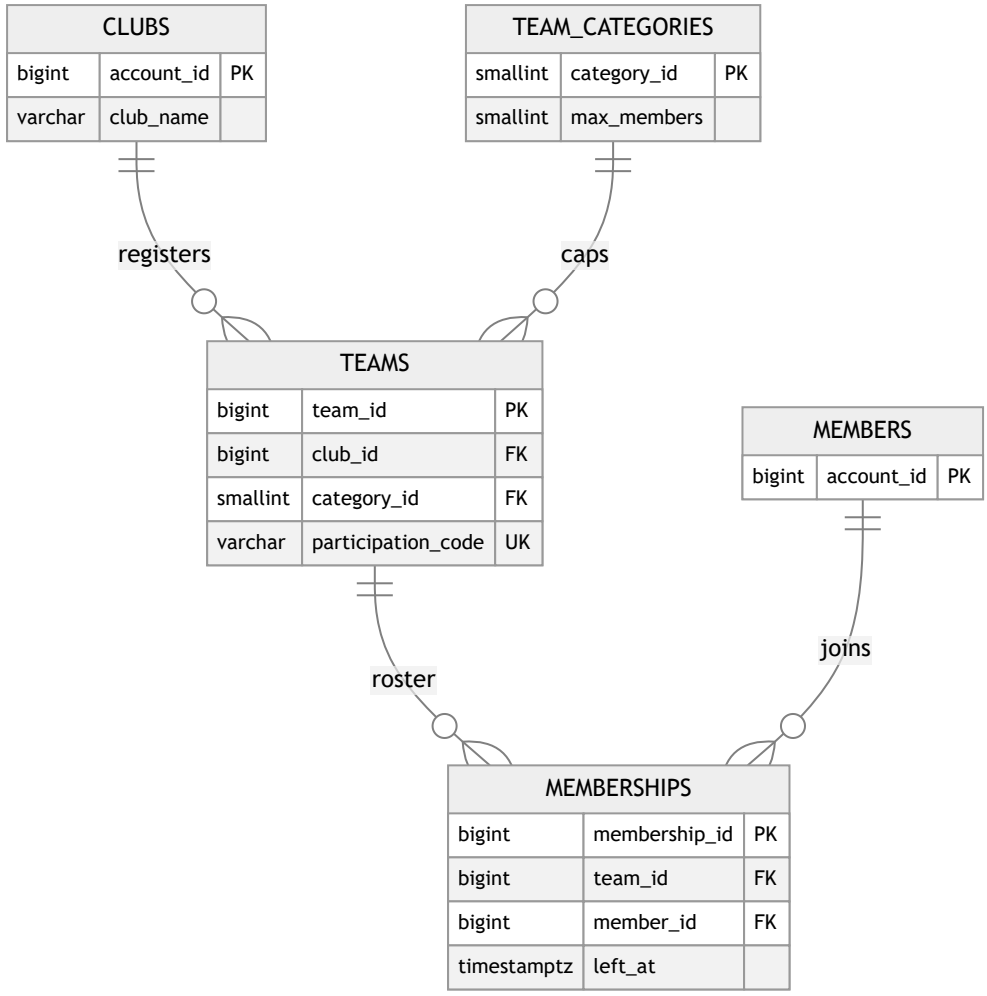
CREATE TABLE clubs (
  account_id       BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  club_name        VARCHAR(150) NOT NULL,
  city             VARCHAR(120),
  coach_name       VARCHAR(150),
  founding_number  INT GENERATED ALWAYS AS IDENTITY, -- own sequence, always unique
  notes            TEXT,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_clubs_name UNIQUE (club_name),
  CONSTRAINT uq_clubs_founding_number UNIQUE (founding_number)
);

CREATE TABLE judges (
  account_id  BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  full_name   VARCHAR(150) NOT NULL,
  expertise   VARCHAR(200),
  bio         TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE partners (
  account_id    BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  org_name      VARCHAR(150),
  contact_name  VARCHAR(150) NOT NULL,
  partner_type  partner_type NOT NULL, -- distinguishes Partner vs Speaker
  message       TEXT,
```

```
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3 Clubs & Teams — Category caps as data, not code; members↔teams via memberships.



memberships is the resolved many-to-many between members and teams; category caps travel with team_categories, not a hardcoded rule.

```
CREATE TABLE team_categories (
  category_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code         VARCHAR(20) NOT NULL,
  label        VARCHAR(60) NOT NULL,
  max_members  SMALLINT NOT NULL CHECK (max_members > 0),
  CONSTRAINT uq_team_categories_code UNIQUE (code)
);

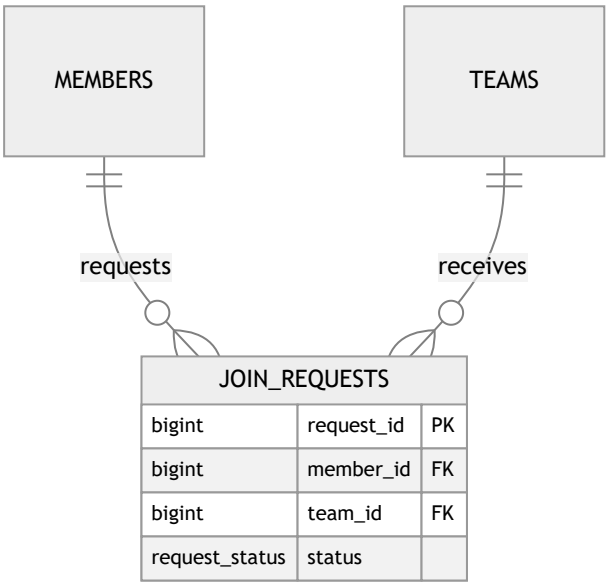
INSERT INTO team_categories (code, label, max_members) VALUES
  ('ROBOTICS',    'Robotics',    5),
  ('PROGRAMMING', 'Programming', 3),
  ('DESIGN',      'Design',      5),
  ('OTHER',       'Other',       5);

CREATE TABLE teams (
  team_id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  club_id             BIGINT NOT NULL REFERENCES clubs(account_id) ON DELETE CASCADE,
  category_id         SMALLINT NOT NULL REFERENCES team_categories(category_id),
  team_name           VARCHAR(150) NOT NULL,
  participation_code  VARCHAR(20) NOT NULL,
  is_active           BOOLEAN NOT NULL DEFAULT TRUE,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_teams_participation_code UNIQUE (participation_code),
  CONSTRAINT uq_teams_club_name UNIQUE (club_id, team_name)
);
```

```sql
CREATE INDEX ix_teams_club_id ON teams(club_id);
CREATE INDEX ix_teams_category_id ON teams(category_id);

CREATE TABLE memberships (
  membership_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id        BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  member_id      BIGINT NOT NULL REFERENCES members(account_id) ON DELETE CASCADE,
  role_title     VARCHAR(60),
  joined_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  left_at        TIMESTAMPTZ -- NULL while active
);
CREATE INDEX ix_memberships_team_id ON memberships(team_id);
CREATE INDEX ix_memberships_member_id ON memberships(member_id);
CREATE UNIQUE INDEX uq_memberships_active_team_member
  ON memberships(team_id, member_id) WHERE left_at IS NULL;
```

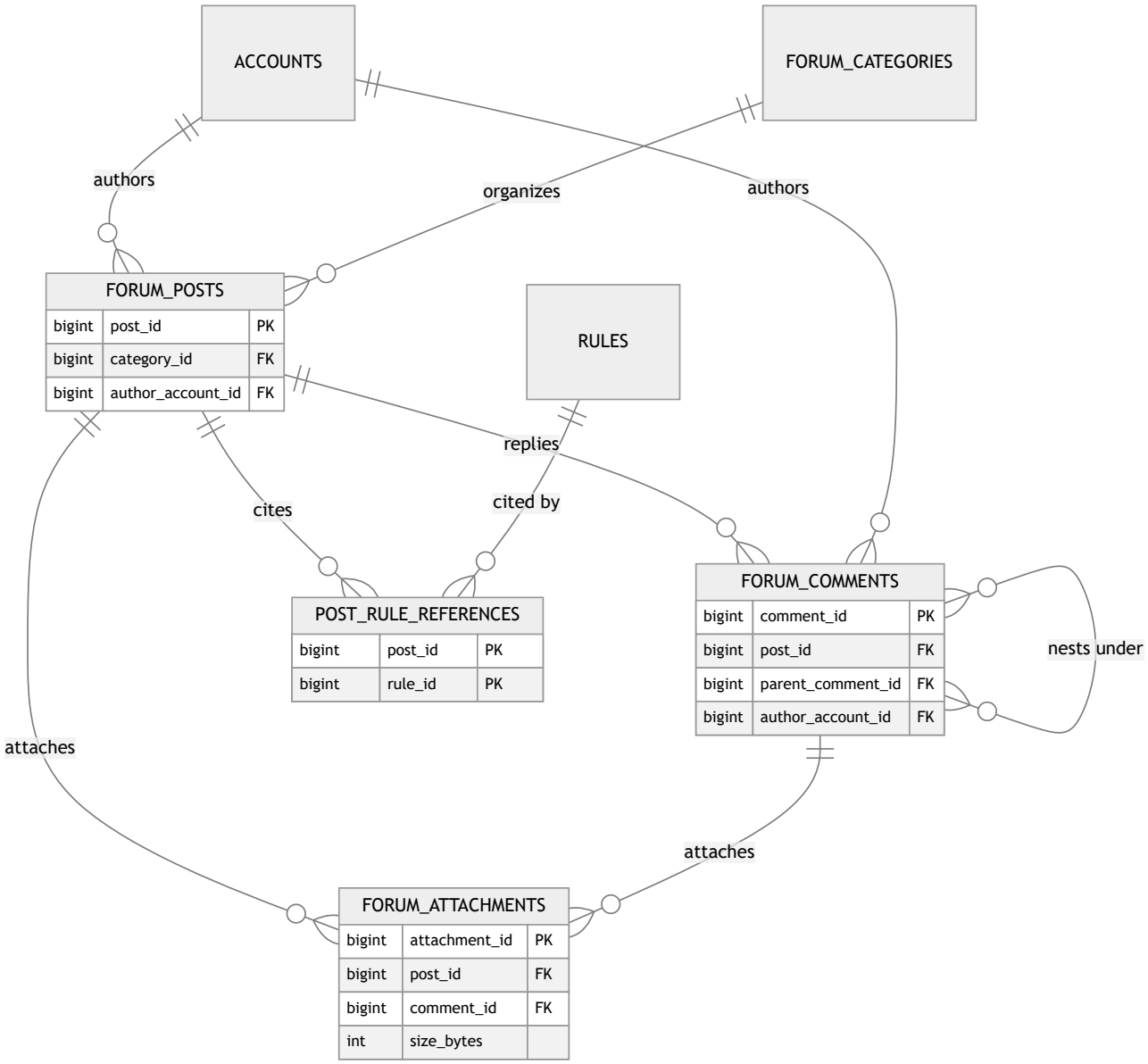
4 Join Requests — Partial unique index allows re-requesting after a rejection.



A request links a member to a team; who gets notified about it lives in the Notifications section at the end.

```sql
CREATE TABLE join_requests (
  request_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id     BIGINT NOT NULL REFERENCES members(account_id) ON DELETE CASCADE,
  team_id       BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  status        request_status NOT NULL DEFAULT 'pending',
  message       TEXT,
  requested_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  decided_at    TIMESTAMPTZ,
  decided_by    BIGINT REFERENCES accounts(account_id)
);
CREATE INDEX ix_join_requests_member_id ON join_requests(member_id);
CREATE INDEX ix_join_requests_team_id ON join_requests(team_id);
CREATE INDEX ix_join_requests_decided_by ON join_requests(decided_by);
CREATE UNIQUE INDEX uq_join_requests_pending
  ON join_requests(member_id, team_id) WHERE status = 'pending';
```

5 **Forum** — Post type (member/club) is derived from the author's account role, never stored.



Attachments hang off a post or a comment (never both); post_rule_references is the real junction table behind the post↔rule many-to-many.

```
CREATE TABLE forum_categories (
  category_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code         VARCHAR(30) NOT NULL,
  label        VARCHAR(60) NOT NULL,
  sort_order   SMALLINT NOT NULL DEFAULT 0,
  CONSTRAINT uq_forum_categories_code UNIQUE (code)
);

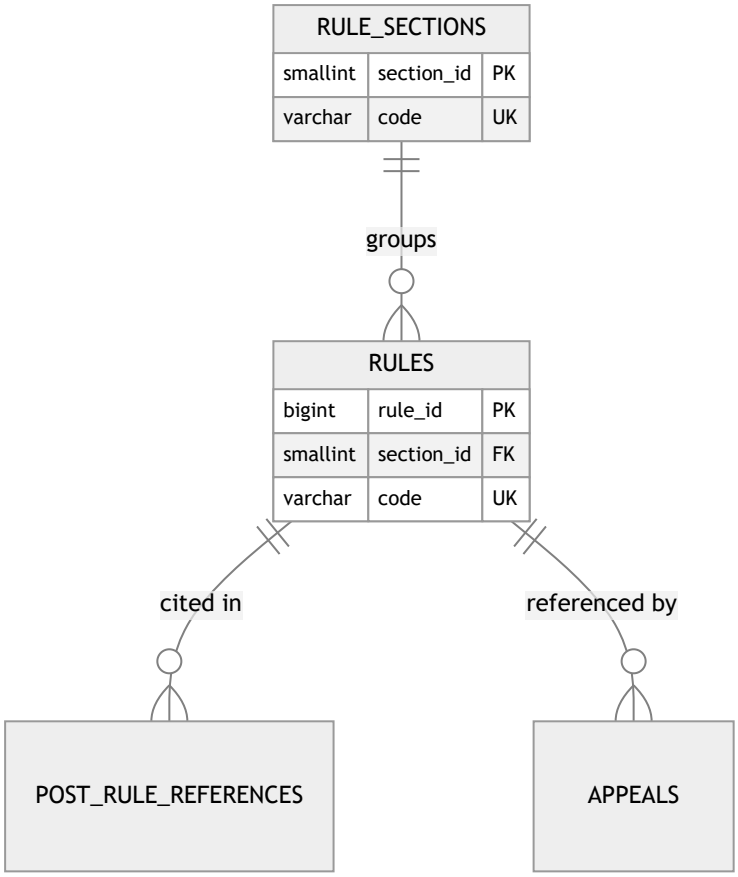
CREATE TABLE forum_posts (
  post_id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  category_id        SMALLINT NOT NULL REFERENCES forum_categories(category_id),
  author_account_id  BIGINT NOT NULL REFERENCES accounts(account_id),
  title              VARCHAR(200) NOT NULL,
  body               TEXT,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  edited_at          TIMESTAMPTZ,
  is_deleted         BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX ix_forum_posts_category_id ON forum_posts(category_id);
```

```sql
CREATE INDEX ix_forum_posts_author_id ON forum_posts(author_account_id);

CREATE TABLE forum_comments (
  comment_id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id            BIGINT NOT NULL REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  parent_comment_id  BIGINT REFERENCES forum_comments(comment_id) ON DELETE CASCADE,
  author_account_id  BIGINT NOT NULL REFERENCES accounts(account_id),
  body               TEXT NOT NULL,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  edited_at          TIMESTAMPTZ,
  is_deleted         BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX ix_forum_comments_post_id ON forum_comments(post_id);
CREATE INDEX ix_forum_comments_parent_id ON forum_comments(parent_comment_id);
CREATE INDEX ix_forum_comments_author_id ON forum_comments(author_account_id);

CREATE TABLE forum_attachments (
  attachment_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id        BIGINT REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  comment_id     BIGINT REFERENCES forum_comments(comment_id) ON DELETE CASCADE,
  file_name      VARCHAR(255) NOT NULL,
  mime_type      VARCHAR(100) NOT NULL,
  size_bytes     INTEGER NOT NULL CHECK (size_bytes > 0 AND size_bytes ≤ 1048576), -- ≤ 1 MB
  uploaded_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_forum_attachments_one_parent CHECK (num_nonnulls(post_id, comment_id) = 1)
);
CREATE INDEX ix_forum_attachments_post_id ON forum_attachments(post_id);
CREATE INDEX ix_forum_attachments_comment_id ON forum_attachments(comment_id);
```

6 Regulations & Rules — Citable codes (G4, A21, E2…); posts cite rules many-to-many.



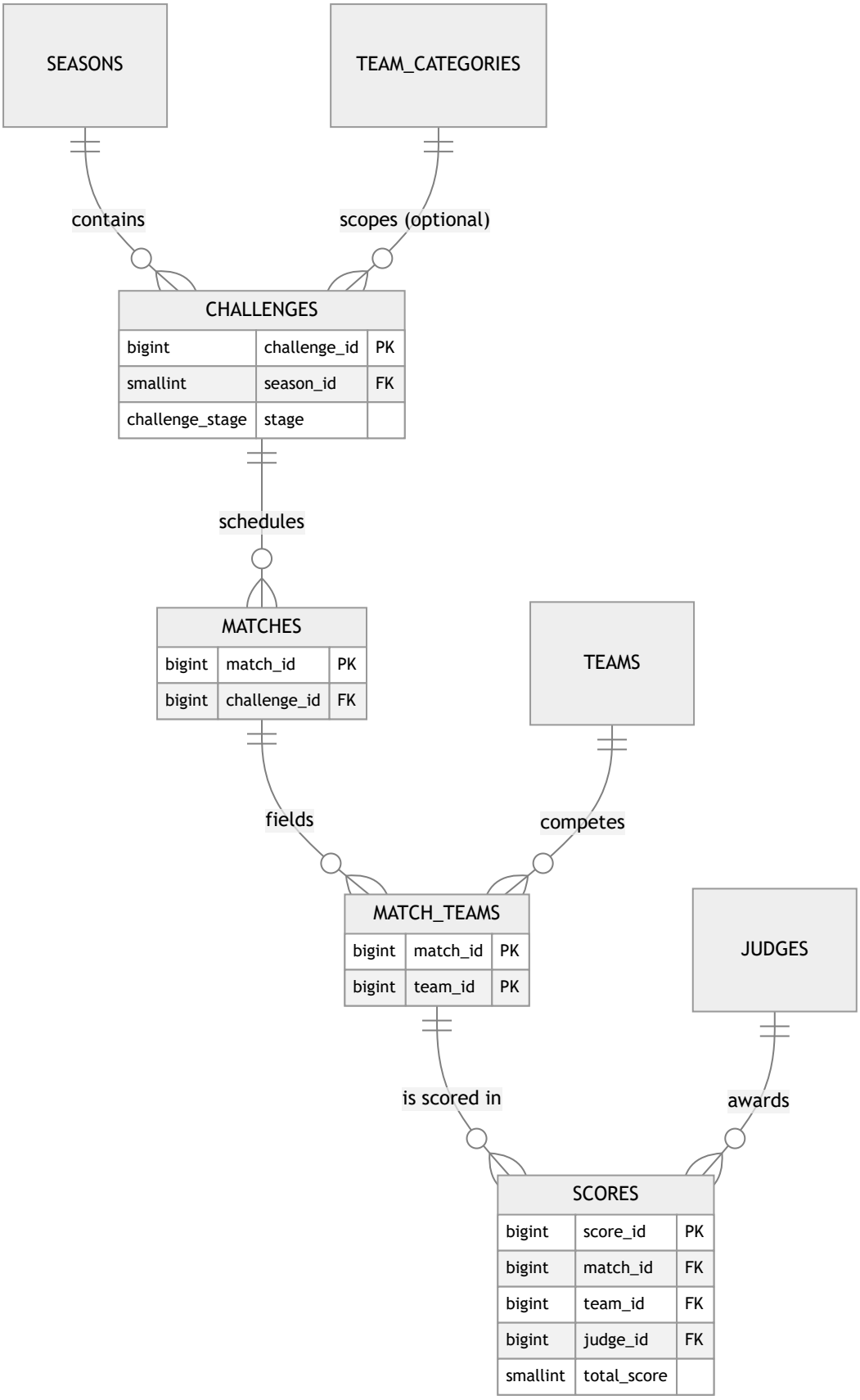
Rules sit at a hub: the forum cites them, and appeals optionally reference one directly.

```
CREATE TABLE rule_sections (
  section_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code        VARCHAR(5) NOT NULL, -- 'G', 'S', 'A', 'E'
  title       VARCHAR(100) NOT NULL,
  sort_order  SMALLINT NOT NULL DEFAULT 0,
  CONSTRAINT uq_rule_sections_code UNIQUE (code)
);

CREATE TABLE rules (
  rule_id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  section_id     SMALLINT NOT NULL REFERENCES rule_sections(section_id),
  code           VARCHAR(10) NOT NULL, -- 'G4', 'A21', 'E2'…
  title          VARCHAR(200) NOT NULL,
  body           TEXT NOT NULL,
  version_label  VARCHAR(30),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_rules_code UNIQUE (code)
);
CREATE INDEX ix_rules_section_id ON rules(section_id);

CREATE TABLE post_rule_references (
  post_id     BIGINT NOT NULL REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  rule_id     BIGINT NOT NULL REFERENCES rules(rule_id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (post_id, rule_id)
);
CREATE INDEX ix_post_rule_references_rule_id ON post_rule_references(rule_id);
```

7 Competition — Seasons → challenges → matches → scores, with a generated total.



A straight chain from season down to score; match_teams is the fielding roster a score can only reference if the team is actually in that match.

```sql
CREATE TABLE seasons (
  season_id   SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        VARCHAR(60) NOT NULL,
  start_date  DATE NOT NULL,
  end_date    DATE,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_seasons_name UNIQUE (name)
);

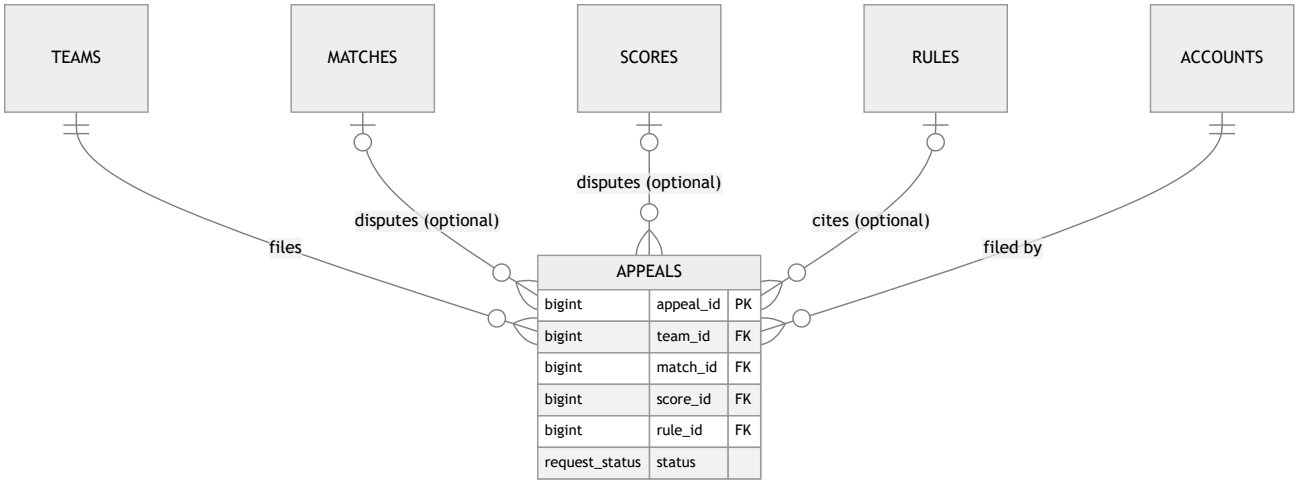
CREATE TABLE challenges (
  challenge_id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  season_id            SMALLINT NOT NULL REFERENCES seasons(season_id),
  category_id          SMALLINT REFERENCES team_categories(category_id), -- NULL = all categories
  name                 VARCHAR(150) NOT NULL,
  description          TEXT,
  stage                challenge_stage NOT NULL,
  submission_deadline  TIMESTAMPTZ
);
CREATE INDEX ix_challenges_season_id ON challenges(season_id);
CREATE INDEX ix_challenges_category_id ON challenges(category_id);

CREATE TABLE matches (
  match_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  challenge_id  BIGINT NOT NULL REFERENCES challenges(challenge_id) ON DELETE CASCADE,
  round_name    VARCHAR(100),
  scheduled_at  TIMESTAMPTZ,
  location      VARCHAR(150),
  status        match_status NOT NULL DEFAULT 'scheduled'
);
CREATE INDEX ix_matches_challenge_id ON matches(challenge_id);

CREATE TABLE match_teams (
  match_id  BIGINT NOT NULL REFERENCES matches(match_id) ON DELETE CASCADE,
  team_id   BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  PRIMARY KEY (match_id, team_id)
);
CREATE INDEX ix_match_teams_team_id ON match_teams(team_id);

CREATE TABLE scores (
  score_id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  match_id            BIGINT NOT NULL,
  team_id             BIGINT NOT NULL,
  judge_id            BIGINT NOT NULL REFERENCES judges(account_id),
  innovation_score    SMALLINT NOT NULL CHECK (innovation_score BETWEEN 1 AND 10),
  execution_score     SMALLINT NOT NULL CHECK (execution_score BETWEEN 1 AND 10),
  presentation_score  SMALLINT NOT NULL CHECK (presentation_score BETWEEN 1 AND 10),
  teamwork_score      SMALLINT NOT NULL CHECK (teamwork_score BETWEEN 1 AND 10),
  total_score         SMALLINT GENERATED ALWAYS AS
                        (innovation_score + execution_score + presentation_score + teamwork_score) STORED,
  comment             TEXT,
  scored_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT fk_scores_match_team FOREIGN KEY (match_id, team_id)
    REFERENCES match_teams(match_id, team_id) ON DELETE CASCADE,
  CONSTRAINT uq_scores_match_team_judge UNIQUE (match_id, team_id, judge_id)
);
CREATE INDEX ix_scores_judge_id ON scores(judge_id);
CREATE INDEX ix_scores_team_id ON scores(team_id);
CREATE INDEX ix_scores_match_id ON scores(match_id);
```

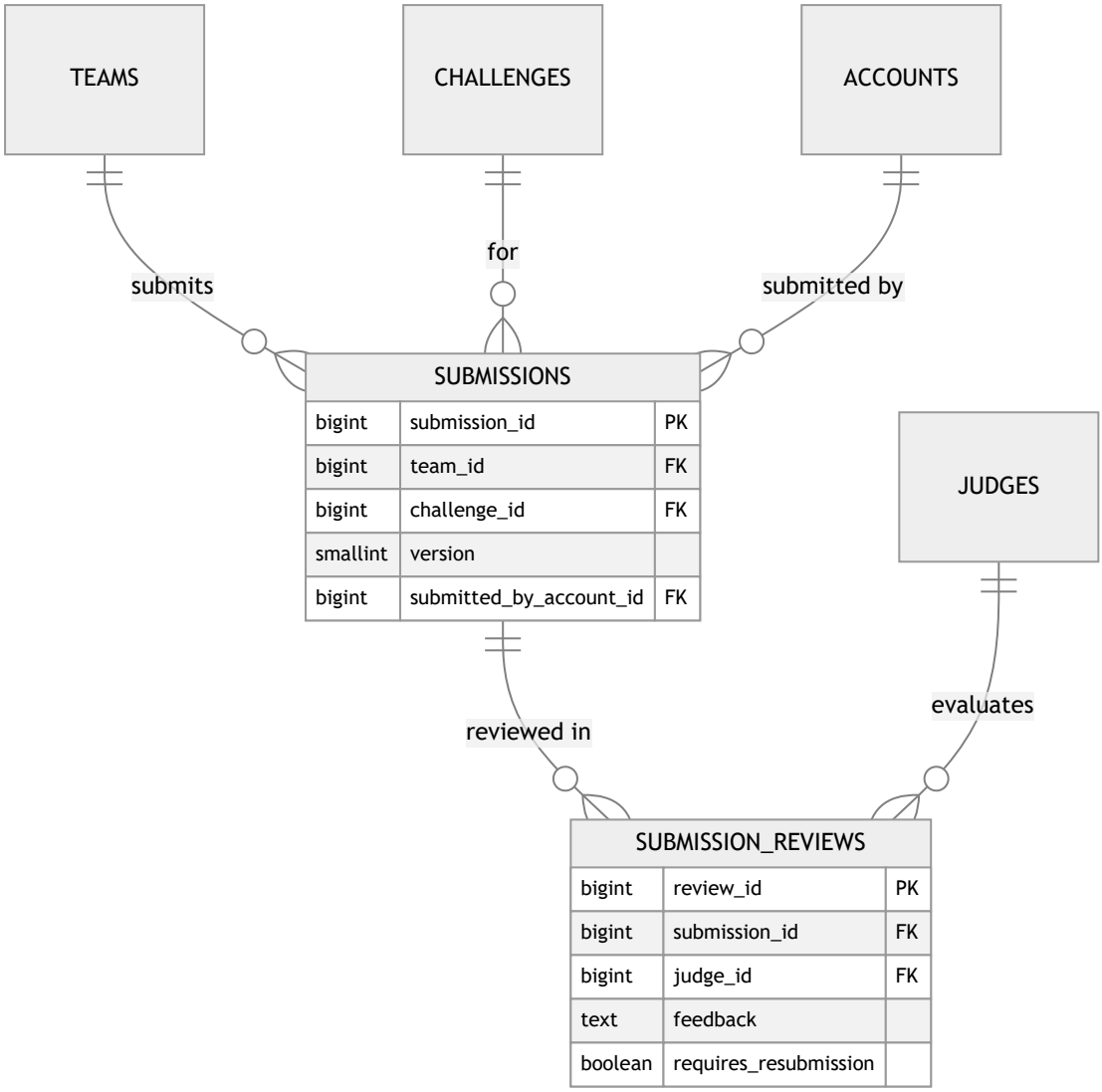
8 Appeals — Max 3 rejected appeals per team, enforced by trigger.



One required link (team), three optional ones (match, score, rule) — an appeal can be as specific or as general as the filer makes it.

```sql
CREATE TABLE appeals (
  appeal_id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id                BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  match_id               BIGINT REFERENCES matches(match_id),
  score_id               BIGINT REFERENCES scores(score_id),
  rule_id                BIGINT REFERENCES rules(rule_id),
  filed_by_account_id    BIGINT NOT NULL REFERENCES accounts(account_id),
  livestream_timestamp   INTERVAL, -- e.g. '01:42:30'
  reason                 TEXT NOT NULL,
  status                 request_status NOT NULL DEFAULT 'pending',
  decided_at             TIMESTAMPTZ,
  decided_by_account_id  BIGINT REFERENCES accounts(account_id),
  created_at             TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_appeals_team_id ON appeals(team_id);
CREATE INDEX ix_appeals_match_id ON appeals(match_id);
CREATE INDEX ix_appeals_score_id ON appeals(score_id);
CREATE INDEX ix_appeals_rule_id ON appeals(rule_id);
CREATE INDEX ix_appeals_filed_by ON appeals(filed_by_account_id);
CREATE INDEX ix_appeals_decided_by ON appeals(decided_by_account_id);
```

9 **Submissions** — Versioned for resubmission; judges connect to a submission via submission_reviews.



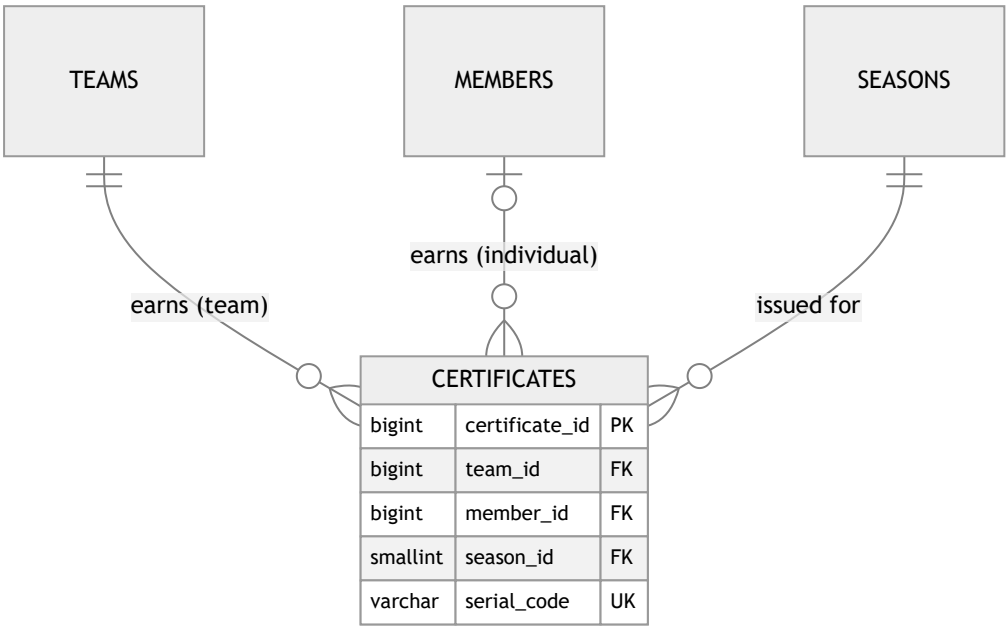
submission_reviews is both the assignment ("this judge evaluates this submission") and the feedback; a resubmission is just a new submissions row (new version) with its own review.

```
CREATE TABLE submissions (
  submission_id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id                  BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  challenge_id             BIGINT NOT NULL REFERENCES challenges(challenge_id) ON DELETE CASCADE,
  version                  SMALLINT NOT NULL DEFAULT 1,
  title                    VARCHAR(200) NOT NULL,
  description              TEXT,
  link_url                 TEXT,
  submitted_by_account_id  BIGINT NOT NULL REFERENCES accounts(account_id),
  created_at               TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at               TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_submissions_team_challenge_version UNIQUE (team_id, challenge_id, version)
);
CREATE INDEX ix_submissions_challenge_id ON submissions(challenge_id);
CREATE INDEX ix_submissions_submitted_by ON submissions(submitted_by_account_id);

CREATE TABLE submission_reviews (
  review_id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  submission_id          BIGINT NOT NULL REFERENCES submissions(submission_id) ON DELETE CASCADE,
  judge_id               BIGINT NOT NULL REFERENCES judges(account_id) ON DELETE CASCADE,
  feedback                TEXT,
```

```
  requires_resubmission   BOOLEAN NOT NULL DEFAULT FALSE,
  reviewed_at             TIMESTAMPTZ,
  assigned_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_submission_reviews_submission_judge UNIQUE (submission_id, judge_id)
);
CREATE INDEX ix_submission_reviews_submission_id ON submission_reviews(submission_id);
CREATE INDEX ix_submission_reviews_judge_id ON submission_reviews(judge_id);
```

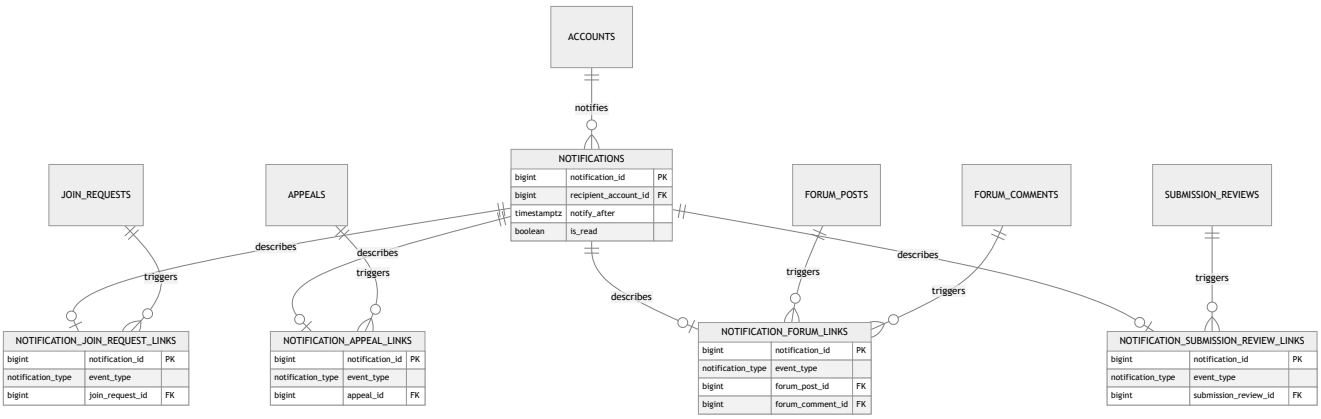
10 Certificates — One table; NULL member_id = team-level certificate.



member_id is optional on purpose: absent it's a team certificate, present it's that one participant's.

```
CREATE TABLE certificates (
  certificate_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id           BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  member_id         BIGINT REFERENCES members(account_id) ON DELETE CASCADE, -- NULL = team-level
  season_id         SMALLINT NOT NULL REFERENCES seasons(season_id),
  certificate_type  certificate_type NOT NULL DEFAULT 'participation',
  serial_code       VARCHAR(40) NOT NULL,
  file_url          TEXT,
  issued_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_certificates_serial UNIQUE (serial_code)
);
CREATE INDEX ix_certificates_team_id ON certificates(team_id);
CREATE INDEX ix_certificates_member_id ON certificates(member_id);
CREATE INDEX ix_certificates_season_id ON certificates(season_id);
CREATE UNIQUE INDEX uq_certificates_team_level
  ON certificates(team_id, season_id, certificate_type) WHERE member_id IS NULL;
CREATE UNIQUE INDEX uq_certificates_participant_level
  ON certificates(team_id, member_id, season_id, certificate_type) WHERE member_id IS NOT NULL;
```

11 Notifications — One destination table + four separate, domain-scoped link tables — unrelated domains never share a table.



Forum activity and appeals have nothing to do with each other, so notification_forum_links and notification_appeal_links are separate tables, not shared columns on one wide table.

```
CREATE TABLE notifications (
  notification_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  recipient_account_id  BIGINT NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
  title                 VARCHAR(150) NOT NULL,
  body                  TEXT,
  notify_after          TIMESTAMPTZ NOT NULL DEFAULT now(), -- delivery buffer, e.g. now()+30min for
comments/replies
  is_read               BOOLEAN NOT NULL DEFAULT FALSE,
  created_at            TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_notifications_recipient ON notifications(recipient_account_id, is_read);
CREATE INDEX ix_notifications_notify_after ON notifications(notify_after) WHERE is_read = FALSE;

CREATE TABLE notification_join_request_links (
  notification_id  BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type       notification_type NOT NULL,
  join_request_id  BIGINT NOT NULL REFERENCES join_requests(request_id) ON DELETE CASCADE
);
CREATE INDEX ix_notification_join_request_links_request ON notification_join_request_links(join_request_id);

CREATE TABLE notification_appeal_links (
  notification_id  BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type       notification_type NOT NULL,
  appeal_id        BIGINT NOT NULL REFERENCES appeals(appeal_id) ON DELETE CASCADE
);
CREATE INDEX ix_notification_appeal_links_appeal ON notification_appeal_links(appeal_id);

CREATE TABLE notification_forum_links (
  notification_id   BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type        notification_type NOT NULL,
  forum_post_id     BIGINT REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  forum_comment_id  BIGINT REFERENCES forum_comments(comment_id) ON DELETE CASCADE,
  CONSTRAINT ck_notification_forum_links_one_source CHECK (num_nonnulls(forum_post_id, forum_comment_id) = 1)
);
CREATE INDEX ix_notification_forum_links_post ON notification_forum_links(forum_post_id);
CREATE INDEX ix_notification_forum_links_comment ON notification_forum_links(forum_comment_id);

CREATE TABLE notification_submission_review_links (
  notification_id      BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type           notification_type NOT NULL,
  submission_review_id  BIGINT NOT NULL REFERENCES submission_reviews(review_id) ON DELETE CASCADE
);
CREATE INDEX ix_notification_submission_review_links_review ON
notification_submission_review_links(submission_review_id);
```

12 Triggers — Cross-row rules plain CHECK constraints can't express.

```
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS trigger AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_accounts_updated_at BEFORE UPDATE ON accounts
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_submissions_updated_at BEFORE UPDATE ON submissions
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE OR REPLACE FUNCTION enforce_subtype_role() RETURNS trigger AS $$
DECLARE
  expected_role account_role := TG_ARGV[0]::account_role;
  actual_role account_role;
BEGIN
  SELECT role INTO actual_role FROM accounts WHERE account_id = NEW.account_id;
  IF actual_role IS DISTINCT FROM expected_role THEN
    RAISE EXCEPTION 'account % has role %, expected % for table %',
      NEW.account_id, actual_role, expected_role, TG_TABLE_NAME;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_members_role_check BEFORE INSERT OR UPDATE ON members
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('member');
CREATE TRIGGER trg_clubs_role_check BEFORE INSERT OR UPDATE ON clubs
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('club');
CREATE TRIGGER trg_judges_role_check BEFORE INSERT OR UPDATE ON judges
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('judge');
CREATE TRIGGER trg_partners_role_check BEFORE INSERT OR UPDATE ON partners
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('partner');

CREATE OR REPLACE FUNCTION enforce_team_capacity() RETURNS trigger AS $$
DECLARE
  v_max SMALLINT;
  v_current INT;
BEGIN
  SELECT tc.max_members INTO v_max
  FROM teams t JOIN team_categories tc ON tc.category_id = t.category_id
  WHERE t.team_id = NEW.team_id;

  SELECT count(*) INTO v_current
  FROM memberships WHERE team_id = NEW.team_id AND left_at IS NULL;

  IF v_current ≥ v_max THEN
    RAISE EXCEPTION 'team % is at capacity (% members, category max %)', NEW.team_id, v_current, v_max;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_memberships_capacity BEFORE INSERT ON memberships
  FOR EACH ROW EXECUTE FUNCTION enforce_team_capacity();

CREATE OR REPLACE FUNCTION enforce_appeal_limit() RETURNS trigger AS $$
DECLARE
  v_rejected_count INT;
BEGIN
  SELECT count(*) INTO v_rejected_count
  FROM appeals WHERE team_id = NEW.team_id AND status = 'rejected';

  IF v_rejected_count ≥ 3 THEN
```

```
    RAISE EXCEPTION 'team % has reached the limit of 3 invalid (rejected) appeals', NEW.team_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_appeals_limit BEFORE INSERT ON appeals
  FOR EACH ROW EXECUTE FUNCTION enforce_appeal_limit();

CREATE OR REPLACE FUNCTION assign_submission_version() RETURNS trigger AS $$
BEGIN
  SELECT COALESCE(MAX(version), 0) + 1 INTO NEW.version
  FROM submissions
  WHERE team_id = NEW.team_id AND challenge_id = NEW.challenge_id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_submissions_version BEFORE INSERT ON submissions
  FOR EACH ROW EXECUTE FUNCTION assign_submission_version();

CREATE OR REPLACE FUNCTION enforce_single_notification_link() RETURNS trigger AS $$
DECLARE
  existing_count INT;
BEGIN
  SELECT
    (SELECT count(*) FROM notification_join_request_links WHERE notification_id = NEW.notification_id) +
    (SELECT count(*) FROM notification_appeal_links WHERE notification_id = NEW.notification_id) +
    (SELECT count(*) FROM notification_forum_links WHERE notification_id = NEW.notification_id) +
    (SELECT count(*) FROM notification_submission_review_links WHERE notification_id = NEW.notification_id)
  INTO existing_count;

  IF existing_count > 0 THEN
    RAISE EXCEPTION 'notification % already has a link', NEW.notification_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_notification_join_request_links_single BEFORE INSERT ON notification_join_request_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
CREATE TRIGGER trg_notification_appeal_links_single BEFORE INSERT ON notification_appeal_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
CREATE TRIGGER trg_notification_forum_links_single BEFORE INSERT ON notification_forum_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
CREATE TRIGGER trg_notification_submission_review_links_single BEFORE INSERT ON notification_submission_review_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
```

13 **Row-Level Security (optional)** — DB-layer illustration of the member/club submission-edit rule.

```sql
ALTER TABLE submissions ENABLE ROW LEVEL SECURITY;

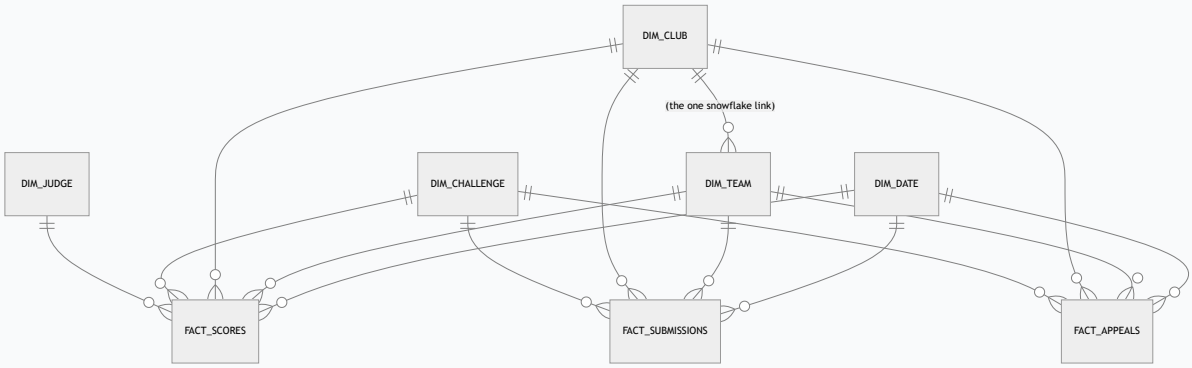
CREATE POLICY submissions_team_member_edit ON submissions
  FOR UPDATE
  USING (
    current_setting('app.current_role', true) = 'member'
    AND team_id IN (
      SELECT m.team_id FROM memberships m
      WHERE m.member_id = current_setting('app.current_account_id', true)::BIGINT
        AND m.left_at IS NULL
    )
  );

CREATE POLICY submissions_club_edit ON submissions
  FOR UPDATE
  USING (
    current_setting('app.current_role', true) = 'club'
    AND team_id IN (
      SELECT t.team_id FROM teams t
      WHERE t.club_id = current_setting('app.current_account_id', true)::BIGINT
    )
  );
```

ANALYTICS SCHEMA star schema

The star

Three fact tables, each reaching the same five dimensions — the shape a partner-dashboard query actually walks.



1 **Dimensions** — dim_date, dim_club, dim_team, dim_challenge, dim_judge.

```
CREATE SCHEMA IF NOT EXISTS analytics;

CREATE TABLE analytics.dim_date (
  date_key     INT PRIMARY KEY, -- YYYYMMDD
  full_date    DATE NOT NULL,
  day          SMALLINT NOT NULL,
  month        SMALLINT NOT NULL,
  month_name   VARCHAR(20) NOT NULL,
  quarter      SMALLINT NOT NULL,
  year         SMALLINT NOT NULL,
  day_of_week  SMALLINT NOT NULL,
  day_name     VARCHAR(20) NOT NULL,
  is_weekend   BOOLEAN NOT NULL,
  CONSTRAINT uq_dim_date_full_date UNIQUE (full_date)
);

CREATE TABLE analytics.dim_club (
  club_key         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  club_id          BIGINT NOT NULL, -- OLTP clubs.account_id
  club_name        VARCHAR(150) NOT NULL,
  city             VARCHAR(120),
  founding_number  INT,
  CONSTRAINT uq_dim_club_club_id UNIQUE (club_id)
);

CREATE TABLE analytics.dim_team (
  team_key            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id             BIGINT NOT NULL, -- OLTP teams.team_id
  club_key            BIGINT NOT NULL REFERENCES analytics.dim_club(club_key),
  team_name           VARCHAR(150) NOT NULL,
  participation_code  VARCHAR(20) NOT NULL,
  category_code       VARCHAR(20) NOT NULL,
  category_label      VARCHAR(60) NOT NULL,
  CONSTRAINT uq_dim_team_team_id UNIQUE (team_id)
);
CREATE INDEX ix_dim_team_club_key ON analytics.dim_team(club_key);

CREATE TABLE analytics.dim_challenge (
  challenge_key   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  challenge_id    BIGINT NOT NULL, -- OLTP challenges.challenge_id
  challenge_name  VARCHAR(150) NOT NULL,
  stage           VARCHAR(30) NOT NULL,
  season_name     VARCHAR(60) NOT NULL,
  category_label  VARCHAR(60),
  CONSTRAINT uq_dim_challenge_challenge_id UNIQUE (challenge_id)
);

CREATE TABLE analytics.dim_judge (
  judge_key   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  judge_id    BIGINT NOT NULL, -- OLTP judges.account_id
  judge_name  VARCHAR(150) NOT NULL,
  expertise   VARCHAR(200),
  CONSTRAINT uq_dim_judge_judge_id UNIQUE (judge_id)
);
```

2 Fact Tables — fact_scores, fact_submissions, fact_appeals — one row per event.

```sql
CREATE TABLE analytics.fact_scores (
  fact_score_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  date_key            INT NOT NULL REFERENCES analytics.dim_date(date_key),
  team_key            BIGINT NOT NULL REFERENCES analytics.dim_team(team_key),
  club_key            BIGINT NOT NULL REFERENCES analytics.dim_club(club_key),
  challenge_key       BIGINT NOT NULL REFERENCES analytics.dim_challenge(challenge_key),
  judge_key           BIGINT NOT NULL REFERENCES analytics.dim_judge(judge_key),
  innovation_score    SMALLINT NOT NULL,
  execution_score     SMALLINT NOT NULL,
  presentation_score  SMALLINT NOT NULL,
  teamwork_score      SMALLINT NOT NULL,
  total_score         SMALLINT NOT NULL, -- SUM/AVG per team_key = the leaderboard
  source_score_id     BIGINT NOT NULL,
  CONSTRAINT uq_fact_scores_source UNIQUE (source_score_id)
);
CREATE INDEX ix_fact_scores_date_key ON analytics.fact_scores(date_key);
CREATE INDEX ix_fact_scores_team_key ON analytics.fact_scores(team_key);
CREATE INDEX ix_fact_scores_club_key ON analytics.fact_scores(club_key);
CREATE INDEX ix_fact_scores_challenge_key ON analytics.fact_scores(challenge_key);
CREATE INDEX ix_fact_scores_judge_key ON analytics.fact_scores(judge_key);

CREATE TABLE analytics.fact_submissions (
  fact_submission_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  date_key              INT NOT NULL REFERENCES analytics.dim_date(date_key),
  team_key              BIGINT NOT NULL REFERENCES analytics.dim_team(team_key),
  club_key              BIGINT NOT NULL REFERENCES analytics.dim_club(club_key),
  challenge_key         BIGINT NOT NULL REFERENCES analytics.dim_challenge(challenge_key),
  submission_count      SMALLINT NOT NULL DEFAULT 1, -- "factless fact": SUM() = count
  source_submission_id  BIGINT NOT NULL,
  CONSTRAINT uq_fact_submissions_source UNIQUE (source_submission_id)
);
CREATE INDEX ix_fact_submissions_date_key ON analytics.fact_submissions(date_key);
CREATE INDEX ix_fact_submissions_team_key ON analytics.fact_submissions(team_key);
CREATE INDEX ix_fact_submissions_club_key ON analytics.fact_submissions(club_key);
CREATE INDEX ix_fact_submissions_challenge_key ON analytics.fact_submissions(challenge_key);

CREATE TABLE analytics.fact_appeals (
  fact_appeal_id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  date_key              INT NOT NULL REFERENCES analytics.dim_date(date_key),
  team_key              BIGINT NOT NULL REFERENCES analytics.dim_team(team_key),
  club_key              BIGINT NOT NULL REFERENCES analytics.dim_club(club_key),
  challenge_key         BIGINT REFERENCES analytics.dim_challenge(challenge_key),
  appeal_status         VARCHAR(20) NOT NULL, -- snapshot at ETL time
  valid_appeal_count    SMALLINT NOT NULL DEFAULT 0, -- 1 if pending/accepted
  invalid_appeal_count  SMALLINT NOT NULL DEFAULT 0, -- 1 if rejected
  source_appeal_id      BIGINT NOT NULL,
  CONSTRAINT uq_fact_appeals_source UNIQUE (source_appeal_id),
  CONSTRAINT ck_fact_appeals_flags CHECK (valid_appeal_count + invalid_appeal_count = 1)
);
CREATE INDEX ix_fact_appeals_date_key ON analytics.fact_appeals(date_key);
CREATE INDEX ix_fact_appeals_team_key ON analytics.fact_appeals(team_key);
CREATE INDEX ix_fact_appeals_club_key ON analytics.fact_appeals(club_key);
CREATE INDEX ix_fact_appeals_challenge_key ON analytics.fact_appeals(challenge_key);
```

3 Example Views — Leaderboard + season progress, built from the facts above.

```sql
CREATE VIEW analytics.vw_leaderboard AS
SELECT
  t.team_key, t.team_id, t.team_name, t.category_label,
  c.club_key, c.club_name,
  count(*)                      AS scored_matches,
  sum(fs.total_score)           AS total_score,
  round(avg(fs.total_score), 1)  AS avg_score
FROM analytics.fact_scores fs
JOIN analytics.dim_team t ON t.team_key = fs.team_key
JOIN analytics.dim_club c ON c.club_key = fs.club_key
GROUP BY t.team_key, t.team_id, t.team_name, t.category_label, c.club_key, c.club_name
ORDER BY avg_score DESC;

CREATE VIEW analytics.vw_team_season_progress AS
WITH season_challenge_counts AS (
  SELECT season_name, count(*) AS total_challenges
  FROM analytics.dim_challenge GROUP BY season_name
),
team_submission_counts AS (
  SELECT t.team_key, t.team_name, ch.season_name,
    count(DISTINCT fsub.challenge_key) AS submitted_challenges
  FROM analytics.dim_team t
  JOIN analytics.fact_submissions fsub ON fsub.team_key = t.team_key
  JOIN analytics.dim_challenge ch ON ch.challenge_key = fsub.challenge_key
  GROUP BY t.team_key, t.team_name, ch.season_name
)
SELECT
  tsc.team_key, tsc.team_name, tsc.season_name,
  tsc.submitted_challenges, scc.total_challenges,
  round(100.0 * tsc.submitted_challenges / NULLIF(scc.total_challenges, 0), 1) AS progress_pct
FROM team_submission_counts tsc
JOIN season_challenge_counts scc ON scc.season_name = tsc.season_name;
```

Condensed reference — see the source .sql files for full inline design-decision comments.